Code :

```
from vertexai.resources.preview import ml_monitoring

my_model_monitor = ml_monitoring.ModelMonitor.create(
  project=PROJECT_ID,
  location=LOCATION,
  display_name="churn_model_monitor",
  model_name=model.resource_name,
  model_version_id="1",
  training_dataset=TRAINING_DATASET,
  model_monitoring_schema=MODEL_MONITORING_SCHEMA,
)
MODEL_MONITOR_ID = my_model_monitor.name
print(f"MODEL MONITOR {MODEL_MONITOR_ID} created.")
```

Line-by-Line Explanation

- **from vertexai.resources.preview import ml_monitoring**: This imports the necessary module from the Vertex AI SDK to manage monitoring resources.
- **ml_monitoring.ModelMonitor.create(...)**: This is the core command that provisions the monitoring job in your Google Cloud project.
    - **project=PROJECT_ID & location=LOCATION**: These define the Google Cloud project and the specific region (e.g., us-central1) where the monitor will be managed.
    - **display_name="churn_model_monitor"**: A human-readable name that will appear in the Google Cloud Console, making it easy to find your specific monitor.
    - **model_name=model.resource_name**: This links the monitor to a specific model already registered in the **Vertex AI Model Registry**. The monitor needs this to know which model's predictions it should be watching.
    - **model_version_id="1"**: This specifies *which version* of that model you want to monitor. Monitoring is version-specific because you might have different versions of a model trained on slightly different features.
    - **training_dataset=TRAINING_DATASET**: **This is crucial.** It tells the monitor what "normal" looks like. The system calculates statistical profiles (mean, standard deviation, etc.) from this dataset to use as a "Golden Baseline" for future comparisons.

    - **model_monitoring_schema=MODEL_MONITORING_SCHEMA**: This tells the system how to interpret your data (e.g., which columns are categorical, which are numerical, and which column is the target variable). Without this schema, the monitor wouldn't know how to mathematically compare the feature distributions.
- **MODEL_MONITOR_ID = my_model_monitor.name**: This stores the unique ID of the created monitor in a variable, which you would use later if you needed to check the status, update configurations, or delete the monitor.

1. Input Feature Drift Detection

This is the most common form of monitoring. It answers the question: **"Has the data being fed into my model changed since I trained it?"**

- **The Concept**: Machine learning models learn patterns based on a specific distribution of data (e.g., a "churn" model trained on data where users typically have 1-5 years of account history). If the real-world input data shifts (e.g., your new customers are now primarily 10+ year veterans), the model's performance will likely drop.

2. Prediction Output Drift Detection

This answers the question: **"Has the actual result (the prediction) my model is giving changed significantly?"**

- **The Concept**: Sometimes, even if your input data looks fine, the output starts to look strange.
- **Example**: Imagine a fraud detection model that usually marks 1% of transactions as "fraud." If, overnight, the model starts marking 20% of transactions as "fraud," there is a massive prediction output drift.
- **Why it matters**: This is a "sanity check." Even if you aren't sure *why* the model is acting up, a sudden spike or drop in your prediction outputs is a strong signal that something is fundamentally broken.

3. Feature Attribution Drift Detection

This is the most advanced check and answers the question: **"Is the model making decisions for the right reasons?"**

- **The Concept**: This relies on **Vertex Explainable AI (XAI)**. XAI looks at a prediction and tells you *which features* contributed to it (e.g., "The model predicted 'Churn' primarily because of 'High Monthly Bill' and 'Low Usage'").

- **What it monitors**: Feature Attribution Drift detects if the "logic" the model uses has shifted.
- **Example**:
  - **Baseline**: Your model usually attributes churn to "Monthly Bill."
  - **Drift**: Suddenly, the model starts attributing churn to "Time of Day" instead of "Monthly Bill."
  - Even if the inputs look normal and the total number of churn predictions seems normal, the **logic** has changed. This is a critical indicator that your model might be relying on "spurious correlations" or is failing to understand the current environment.

Summary Table

Objective	Focus	What it catches
Input Drift	The "Input" (X)	Data in the real world is changing (e.g., user demographics changed).
Prediction Drift	The "Result" (Y)	The model is suddenly predicting much more/less of an outcome than usual.
Attribution Drift	The "Reasoning"	The model has changed *how* it weighs features to make decisions.

Code:

```
from vertexai.resources.preview import ml_monitoring

FEATURE_THRESHOLDS = {
  "country": 0.003,
  "cnt_user_engagement": 0.004,
}

FEATURE_DRIFT_SPEC = ml_monitoring.spec.DataDriftSpec(
  categorical_metric_type="l_infinity",
  numeric_metric_type="jensen_shannon_divergence",
  default_categorical_alert_threshold=0.2,
  default_numeric_alert_threshold=0.3,
  feature_alert_thresholds=FEATURE_THRESHOLDS,
```

)

based on the code provided, here is a line-by-line explanation using pointers:

- **from vertexai.resources.preview import ml_monitoring**: Imports the specific module required to access Vertex AI's newer model monitoring features.
- **FEATURE_THRESHOLDS = { ... }**: Creates a Python dictionary to store specific "sensitivity" settings for your most important data columns.
- **"country": 0.003**: Sets a very strict threshold for the country feature; any statistical shift greater than 0.003 between training and serving data will trigger an alert.
- **"cnt_user_engagement": 0.004**: Sets a similarly strict threshold for user engagement counts, ensuring high precision for this specific variable.
- **FEATURE_DRIFT_SPEC = ml_monitoring.spec.DataDriftSpec(...)**: Defines the configuration object (the "Spec") that tells Vertex AI exactly how to calculate and handle data drift.
- **categorical_metric_type="l_infinity"**: Selects the **L-Infinity** distance metric for categorical data, which is designed to detect significant changes in the frequency of specific categories (like a sudden surge in traffic from a specific country).
- **numeric_metric_type="jensen_shannon_divergence"**: Selects the **Jensen-Shannon Divergence** metric for numerical data, a standard mathematical way to measure how much two probability distributions overlap.
- **default_categorical_alert_threshold=0.2**: Establishes a "baseline" sensitivity for all other categorical features not listed in your dictionary; it is less sensitive than your custom rules to avoid unnecessary alerts.
- **default_numeric_alert_threshold=0.3**: Establishes the "baseline" sensitivity for all other numerical features; an alert fires only if the drift is relatively large (0.3).
- **feature_alert_thresholds=FEATURE_THRESHOLDS**: Links your custom, strict dictionary rules into the main specification, ensuring that "country" and "engagement" use the custom values while everything else uses the defaults.

Code:

```
PREDICTION_OUTPUT_DRIFT_SPEC = ml_monitoring.spec.DataDriftSpec(
  categorical_metric_type="l_infinity",
  numeric_metric_type="jensen_shannon_divergence",
  default_categorical_alert_threshold=0.1,
  default_numeric_alert_threshold=0.1,
)
```

PREDICTION_OUTPUT_DRIFT_SPEC = ml_monitoring.spec.DataDriftSpec(: Initializes a configuration object that focuses on the **model's output** rather than the

input features. This monitors the "prediction" column to see if the model's decisions are shifting significantly over time.

categorical_metric_type="l_infinity": Defines the statistical method for categorical outputs (e.g., predicting "Churn" vs "No Churn"). Using *L-Infinity* ensures that if any single category percentage changes dramatically, the system detects it.

numeric_metric_type="jensen_shannon_divergence": Defines the statistical method for numeric outputs (e.g., predicting a price or a probability score). *Jensen-Shannon Divergence* is used to detect if the distribution of these predicted numbers has "stretched" or shifted.

default_categorical_alert_threshold=0.1: Sets the sensitivity for categorical outcomes. If the distribution of your predictions changes by 10% (0.1) compared to the baseline, the monitor will trigger an alert.

default_numeric_alert_threshold=0.1: Sets the sensitivity for numeric outcomes. Like the categorical setting, this triggers an alert if the prediction distribution shifts by more than 10% from the expected baseline.

Code:

```
FEATURE_ATTRIBUTION_SPEC = ml_monitoring.spec.FeatureAttributionSpec(
  default_alert_threshold=0.0003,
  feature_alert_thresholds={"cnt_ad_reward": 0.0001},
)
```

This code block configures the monitoring of **Feature Attribution**, which tracks how the model "thinks" or reaches its decisions. It ensures that the model continues to weigh features in the same way it did during training.

Here is the line-by-line explanation:

- **FEATURE_ATTRIBUTION_SPEC = ml_monitoring.spec.FeatureAttributionSpec(**: Initializes a specification object specifically designed to monitor Feature Attribution. This uses Vertex Explainable AI (XAI) to track whether the relative importance of your input features remains consistent over time.
- **default_alert_threshold=0.0003**: Sets the global sensitivity level for all features. If the "attribution score" (the influence of a feature on a prediction) for any feature drifts by more than 0.0003 from your training baseline, an alert is triggered.
- **feature_alert_thresholds={"cnt_ad_reward": 0.0001},**: Creates a custom exception rule for the specific feature cnt_ad_reward.

    - By setting the threshold to 0.0001 (which is stricter than the 0.0003 default), you are telling the system: "I am extremely sensitive to changes in how the model uses cnt_ad_reward."
    - If the model suddenly starts relying on this feature significantly more or less than it did at training time, you want to be notified immediately, even if other features remain stable.

Why this is different from Data Drift:

- **Data Drift** looks at the *inputs* (e.g., "Are the values of this feature changing?").
- **Feature Attribution Monitoring** looks at the *logic* (e.g., "Is this feature still as important to the model's decision as it used to be?").

If your data remains the same but your model's reasoning shifts, it could indicate that the model has "learned" to rely on a different set of patterns, which can be a sign of model degradation or a change in the environment.

Code :

```
import os
from vertexai.resources.preview import ml_monitoring
EMAIL = "stripling@google.com"  # @param {type:"string"}
if os.getenv("IS_TESTING"):
  EMAIL = "noreply@google.com"
NOTIFICATION_SPEC = ml_monitoring.spec.NotificationSpec(
  user_emails=[EMAIL],
)
OUTPUT_SPEC = ml_monitoring.spec.OutputSpec(gcs_base_dir=BUCKET_URI)
```

This code block configures **where** to send alert notifications and **where** to save the monitoring logs and reports that the system generates.

Here is the line-by-line explanation:

- **import os**: Imports Python's built-in operating system module, which allows the script to interact with environment variables (e.g., checking if the current environment is a test environment).
- **from vertexai.resources.preview import ml_monitoring**: Imports the necessary classes from the Vertex AI SDK to define monitoring specifications.
- **EMAIL = "stripling@google.com"**: Sets the default email address that will receive system alerts if the model drift or skew is detected.

- **if os.getenv("IS_TESTING"):**: This is a best practice check. It looks for a system environment variable called IS_TESTING. If this code is running in an automated testing environment, it will return True.
- **EMAIL = "noreply@google.com"**: If the code is in "Testing" mode, it overrides your personal email with a dummy address. This prevents you from being spammed with alerts while running automated tests or continuous integration (CI) workflows.
- **NOTIFICATION_SPEC = ml_monitoring.spec.NotificationSpec(user_emails=[EMAIL],)**: Creates the configuration object that tells Vertex AI **how to reach you**. It wraps the EMAIL variable into a format that the Vertex AI notification service can understand.
- **OUTPUT_SPEC = ml_monitoring.spec.OutputSpec(gcs_base_dir=BUCKET_URI)**: Defines **where the raw data and analysis reports are stored**. Vertex AI generates detailed logs of the drift analysis; this setting directs the service to save those results into your designated Google Cloud Storage (GCS) bucket (BUCKET_URI).

Why this is necessary:

- **Notifications**: Without NotificationSpec, the system would detect drift but have nowhere to report it, leaving you unaware of potential issues.
- **Output Storage**: Without OutputSpec, you would lose the detailed statistical logs. By saving these to your GCS bucket, you create a "paper trail" that you can audit later to see *exactly* how the data distributions shifted over time.

Run Model Monitoring Jobs

Example 1: Detect feature drift by comparing a batch prediction job with the training dataset.

Let's first create a batch prediction job.

```
BP_INPUT_URI_1 = f"{BUCKET_URI}/model-monitoring/churn/churn_bp_input_1.jsonl"
! gsutil copy gs://cloud-samples-data/vertex-ai/model-monitoring/churn/churn_bp_input_1.jsonl $BP_INPUT_URI_1
```

Here is the line-by-line explanation:

- **BP_INPUT_URI_1 = f"{BUCKET_URI}/model-monitoring/churn/churn_bp_input_1.jsonl"**:
  - This is a Python line that creates a **string variable** containing the file path for your input data.
  - It uses an "f-string" (formatted string) to combine your BUCKET_URI (defined earlier in the notebook) with a specific file name (churn_bp_input_1.jsonl).

    - This tells the system: "When I run my batch job, look for the data in *this* specific location inside *my* bucket."
- **! gsutil copy gs://cloud-samples-data/vertex-ai/model-monitoring/churn/churn_bp_input_1.jsonl $BP_INPUT_URI_1**:
    - **!**: This symbol tells the Jupyter notebook environment to run the following text as a **system command** (specifically, a shell command) rather than Python code.
    - **gsutil**: This is the Google Cloud command-line tool for interacting with Cloud Storage buckets.
    - **copy**: The command to transfer data from one place to another.
    - **gs://cloud-samples-data/...**: This is the **Source**. It is a public Google-provided dataset containing sample transaction data for you to use in this tutorial.
    - **$BP_INPUT_URI_1**: This is the **Destination**. The $ sign tells the shell to use the variable you defined in the Python line above.

```
batch_prediction_job_1 = model.batch_predict(
  generate_explanation=True,
  job_display_name="bp_example_1",
  instances_format="jsonl",
  machine_type="e2-standard-4",
  gcs_source=[BP_INPUT_URI_1],
  gcs_destination_prefix=f"{BUCKET_URI}/bp_output",
  sync=True,
)
```

This code initiates a **Batch Prediction Job** on Vertex AI. This is a request to the cloud service to take a large file of input data, run your model against it, and save the results (and explanations) to storage.

Here is the line-by-line explanation of the configuration:

- **batch_prediction_job_1 = model.batch_predict(**: This is the main function call. It instructs Vertex AI to start processing data in bulk using the model instance you defined earlier. It assigns the resulting job object to batch_prediction_job_1 so you can monitor its progress.
- **generate_explanation=True,**: This enables Vertex Explainable AI (XAI). It tells the system to not only return the prediction (e.g., "Churn: Yes") but also to calculate *why* the model made that decision (the feature attributions).

- **job_display_name="bp_example_1",**: Assigns a human-readable name to the job. This is the name you will see in the Google Cloud Console dashboard, making it easier to identify this specific run among potentially many others.
- **instances_format="jsonl",**: Specifies the format of your input data files. JSONL (JSON Lines) is the standard format here, where each line in the file represents a single, independent data instance (e.g., one customer's profile).
- **machine_type="e2-standard-4",**: Defines the virtual machine size for the batch job. This is the compute resource (CPU and RAM) that will perform the actual math. Using e2-standard-4 provides a balanced amount of memory and processing power for this task.
- **gcs_source=[BP_INPUT_URI_1],**: Points the job to the location of your input data. This is the URI (Uniform Resource Identifier) for the file you uploaded to Google Cloud Storage in the previous step.
- **gcs_destination_prefix=f"{BUCKET_URI}/bp_output",**: Sets the output directory. Vertex AI will automatically create a folder at this location and store your prediction results and explanation logs there once the job finishes.
- **sync=True,**: This tells your notebook how to behave while waiting. By setting this to True, the script will "block" (pause execution) at this line and wait until the entire batch job is completed before moving to the next line of code. If you set this to False, the code would proceed immediately, and the job would run in the background.

Code

```
import pandas as pd
from vertexai.resources.preview import ml_monitoring

TIMESTAMP = pd.Timestamp.utcnow().strftime("%Y%m%d%H%M%S")
JOB_DISPLAY_NAME = f"churn_model_monitoring_job_{TIMESTAMP}"
TARGET_DATASET = ml_monitoring.spec.MonitoringInput(
  batch_prediction_job=batch_prediction_job_1.resource_name
)
model_monitoring_job_1 = my_model_monitor.run(
  display_name=JOB_DISPLAY_NAME,
  baseline_dataset=TRAINING_DATASET,
  target_dataset=TARGET_DATASET,
  tabular_objective_spec=ml_monitoring.spec.TabularObjective(
    # Input feature drift spec.
    feature_drift_spec=FEATURE_DRIFT_SPEC
```

```
    ),
    notification_spec=NOTIFICATION_SPEC,
    output_spec=OUTPUT_SPEC,
)
```

This code block executes the actual **Model Monitoring Job**. It takes all the specifications you defined earlier (drift thresholds, notification settings, output paths) and applies them to compare your batch prediction results against your training data.
Here is the line-by-line explanation:

- **import pandas as pd**: Imports the Pandas library, used here specifically to generate a precise, current timestamp.
- **from vertexai.resources.preview import ml_monitoring**: Imports the necessary classes to manage the monitoring job execution.
- **TIMESTAMP = pd.Timestamp.utcnow().strftime("%Y%m%d%H%M%S")**: Generates a unique, sortable timestamp (e.g., 202604271538). This ensures that every time you run a job, it has a unique ID, preventing naming collisions.
- **JOB_DISPLAY_NAME = f"churn_model_monitoring_job_{TIMESTAMP}"**: Creates a human-readable, unique name for this specific monitoring execution using the timestamp.
- **TARGET_DATASET = ml_monitoring.spec.MonitoringInput(...)**: This is a critical step. It tells the monitor **which data to check**. It references the batch_prediction_job_1 you just ran. The monitor will look at the inputs and outputs of this specific batch job to see if they deviate from the baseline.
- **model_monitoring_job_1 = my_model_monitor.run(**: This is the "Go" button. It starts the asynchronous monitoring job in the cloud.

Arguments passed to the .run() method:

- **display_name=JOB_DISPLAY_NAME**: Assigns the unique name you created to the monitoring job in the Vertex AI Console.
- **baseline_dataset=TRAINING_DATASET**: Tells the monitor what the "correct" or "normal" data distribution looks like (your original training data).
- **target_dataset=TARGET_DATASET**: Tells the monitor what data to inspect (the data from your recent batch prediction job).
- **tabular_objective_spec=ml_monitoring.spec.TabularObjective(...)**: Configures the monitoring goal. Specifically, it plugs in the FEATURE_DRIFT_SPEC you defined earlier, which contains the logic for detecting drift in your features.

- **notification_spec=NOTIFICATION_SPEC**: Tells the system where to send alerts (your email) if the drift exceeds the thresholds.
- **output_spec=OUTPUT_SPEC**: Tells the system where to store the resulting analysis reports and logs (your GCS bucket).

In summary: This code bridges the gap between *configuration* (the specs you wrote) and *action* (the actual analysis). It compares the "Target" (what the model did today) against the "Baseline" (what the model was trained on) to detect if anything has gone wrong.

Example 2: Detect feature drift and prediction output drift by comparing a batch prediction job with a previous batch prediction job result.

You can set up multiple objectives within a single model monitoring job. All metrics are computed by using the same baseline and target dataset.

Create another batch prediction job and compare it with the batch prediction job created previously.

```
BP_INPUT_URI_2 = f"{BUCKET_URI}/model-monitoring/churn/churn_bp_input_2.jsonl"
! gsutil copy gs://cloud-samples-data/vertex-ai/model-
monitoring/churn/churn_bp_input_2.jsonl $BP_INPUT_URI_2
batch_prediction_job_2 = model.batch_predict(
  job_display_name="bp_example_2",
  instances_format="jsonl",
  machine_type="e2-standard-4",
  gcs_source=[BP_INPUT_URI_2],
  gcs_destination_prefix=f"{BUCKET_URI}/bp_output",
  sync=True,
)
import pandas as pd
from vertexai.resources.preview import ml_monitoring

TIMESTAMP = pd.Timestamp.utcnow().strftime("%Y%m%d%H%M%S")
JOB_DISPLAY_NAME = f"churn_model_monitoring_job_{TIMESTAMP}"
BASELINE_DATASET = ml_monitoring.spec.MonitoringInput(
  batch_prediction_job=batch_prediction_job_1.resource_name
)
TARGET_DATASET = ml_monitoring.spec.MonitoringInput(
  batch_prediction_job=batch_prediction_job_2.resource_name
```

```
)
model_monitoring_job_2 = my_model_monitor.run(
  display_name=JOB_DISPLAY_NAME,
  baseline_dataset=BASELINE_DATASET,
  target_dataset=TARGET_DATASET,
  tabular_objective_spec=ml_monitoring.spec.TabularObjective(
    # Input feature drift spec.
    feature_drift_spec=FEATURE_DRIFT_SPEC,
    # Prediction output drift spec.
    prediction_output_drift_spec=PREDICTION_OUTPUT_DRIFT_SPEC,
  ),
  notification_spec=NOTIFICATION_SPEC,
  output_spec=OUTPUT_SPEC,
)
```

This code demonstrates a powerful monitoring scenario: **comparing two different periods of production data**. Instead of comparing current data to the static "training" dataset, you are comparing data from "Job 1" (the past) to "Job 2" (the present) to see if the environment or user behavior has changed between those two points in time.

Preparing the Second Batch Job

- **BP_INPUT_URI_2 = ...**: Defines the storage path for a second, fresh input file (churn_bp_input_2.jsonl) that represents new, current data.
- **! gsutil copy ...**: Copies this new sample data from a public Google bucket into your own GCS bucket so the Vertex AI job can access it.

Executing the Second Prediction Job

- **batch_prediction_job_2 = model.batch_predict(...)**: Starts the second batch prediction job using the new input data.
- **job_display_name="bp_example_2"**: Labels this job as the second example, distinct from the first.
- **instances_format="jsonl", ...**: Standard parameters (same as Job 1) to ensure the compute environment processes the data exactly like the first job.
- **gcs_source=[BP_INPUT_URI_2]**: Tells the job to process the *second* file you just uploaded.
- **gcs_destination_prefix=f"{BUCKET_URI}/bp_output"**: Stores the results of this second job in the same general bucket, likely in a subdirectory that separates the two jobs.

Setting Up the Comparison (The "A/B" Monitor)

- **import pandas as pd...**: Imports utilities and generates a unique timestamp to keep this monitoring job distinct from previous ones.
- **BASELINE_DATASET = ...**: **This is the key concept.** Instead of using the training set, you point the monitor to batch_prediction_job_1. This effectively says: "Use my results from Job 1 as the 'Normal' or 'Baseline' behavior."
- **TARGET_DATASET = ...**: Points the monitor to batch_prediction_job_2. This says: "Check if my results from Job 2 have drifted away from Job 1."
- **model_monitoring_job_2 = my_model_monitor.run(...)**: Executes the monitor comparing the two jobs.

Applying the Specs

- **feature_drift_spec=FEATURE_DRIFT_SPEC**: Checks if the input values in Job 2 are statistically different from Job 1 (e.g., "Are the users in this batch different from the last batch?").
- **prediction_output_drift_spec=PREDICTION_OUTPUT_DRIFT_SPEC**: Checks if the model's actual predictions changed between the two jobs (e.g., "Is the model predicting more churn today than it did in the last batch?").
- **notification_spec & output_spec**: Reuses your previously defined notification and storage settings to ensure you are alerted and that results are saved.

Why this specific pattern is used

By comparing **Job 1 vs. Job 2**, you are performing **Temporal Monitoring**. This is highly effective if you want to detect "Slow Drift"—where the data changes incrementally over time—rather than just comparing everything back to the original training data. It allows you to see how your production data is evolving week-over-week or day-over-day.

Example 3: Feature attribution drift detection, compares the batch prediction job with a GCS baseline dataset

```
from google.cloud.aiplatform_v1beta1.types import (ExplanationMetadata,
                                                   ExplanationParameters,
                                                   ExplanationSpec)

EXPLANATION_SPEC = ExplanationSpec(
  parameters=ExplanationParameters(
    {"sampled_shapley_attribution": {"path_count": 2}}
  ),
  metadata=ExplanationMetadata(
    inputs={
      "cnt_ad_reward": ExplanationMetadata.InputMetadata(
```

```
    {
      "input_tensor_name": "cnt_ad_reward",
      "encoding": "IDENTITY",
      "modality": "numeric",
    }
  ),
  "cnt_challenge_a_friend": ExplanationMetadata.InputMetadata(
    {
      "input_tensor_name": "cnt_challenge_a_friend",
      "encoding": "IDENTITY",
      "modality": "numeric",
    }
  ),
  "cnt_completed_5_levels": ExplanationMetadata.InputMetadata(
    {
      "input_tensor_name": "cnt_completed_5_levels",
      "encoding": "IDENTITY",
      "modality": "numeric",
    }
  ),
  "cnt_level_complete_quickplay": ExplanationMetadata.InputMetadata(
    {
      "input_tensor_name": "cnt_level_complete_quickplay",
      "encoding": "IDENTITY",
      "modality": "numeric",
    }
  ),
  "cnt_level_end_quickplay": ExplanationMetadata.InputMetadata(
    {
      "input_tensor_name": "cnt_level_end_quickplay",
      "encoding": "IDENTITY",
      "modality": "numeric",
    }
  ),
  "cnt_level_reset_quickplay": ExplanationMetadata.InputMetadata(
    {
      "input_tensor_name": "cnt_level_reset_quickplay",
      "encoding": "IDENTITY",
```

```
        "modality": "numeric",
    }
  ),
  "cnt_level_start_quickplay": ExplanationMetadata.InputMetadata(
    {
        "input_tensor_name": "cnt_level_start_quickplay",
        "encoding": "IDENTITY",
        "modality": "numeric",
    }
  ),
  "cnt_post_score": ExplanationMetadata.InputMetadata(
    {
        "input_tensor_name": "cnt_post_score",
        "encoding": "IDENTITY",
        "modality": "numeric",
    }
  ),
  "cnt_spend_virtual_currency": ExplanationMetadata.InputMetadata(
    {
        "input_tensor_name": "cnt_spend_virtual_currency",
        "encoding": "IDENTITY",
        "modality": "numeric",
    }
  ),
  "cnt_use_extra_steps": ExplanationMetadata.InputMetadata(
    {
        "input_tensor_name": "cnt_use_extra_steps",
        "encoding": "IDENTITY",
        "modality": "numeric",
    }
  ),
  "cnt_user_engagement": ExplanationMetadata.InputMetadata(
    {
        "input_tensor_name": "cnt_user_engagement",
        "encoding": "IDENTITY",
        "modality": "numeric",
    }
  ),
```

```
"country": ExplanationMetadata.InputMetadata(
  {
    "input_tensor_name": "country",
    "encoding": "IDENTITY",
    "modality": "categorical",
  }
),
"dayofweek": ExplanationMetadata.InputMetadata(
  {
    "input_tensor_name": "dayofweek",
    "encoding": "IDENTITY",
    "modality": "numeric",
  }
),
"julianday": ExplanationMetadata.InputMetadata(
  {
    "input_tensor_name": "julianday",
    "encoding": "IDENTITY",
    "modality": "numeric",
  }
),
"language": ExplanationMetadata.InputMetadata(
  {
    "input_tensor_name": "language",
    "encoding": "IDENTITY",
    "modality": "categorical",
  }
),
"month": ExplanationMetadata.InputMetadata(
  {
    "input_tensor_name": "month",
    "encoding": "IDENTITY",
    "modality": "numeric",
  }
),
"operating_system": ExplanationMetadata.InputMetadata(
  {
    "input_tensor_name": "operating_system",
```

```
            "encoding": "IDENTITY",
            "modality": "categorical",
          }
        ),
        "user_pseudo_id": ExplanationMetadata.InputMetadata(
          {
            "input_tensor_name": "user_pseudo_id",
            "encoding": "IDENTITY",
            "modality": "categorical",
          }
        ),
      },
      outputs={
        "churned_probs": ExplanationMetadata.OutputMetadata(
          {"output_tensor_name": "churned_probs"}
        )
      },
    ),
)
FEATURE_ATTRIBUTION_BASELINE_DATASET = (
  f"{BUCKET_URI}/model-monitoring/churn/churn_no_ground_truth.jsonl"
)
! gsutil cp gs://cloud-samples-data/vertex-ai/model-
monitoring/churn/churn_no_ground_truth.jsonl
$FEATURE_ATTRIBUTION_BASELINE_DATASET
import pandas as pd
from vertexai.resources.preview import ml_monitoring

TIMESTAMP = pd.Timestamp.utcnow().strftime("%Y%m%d%H%M%S")
JOB_DISPLAY_NAME = f"churn_model_monitoring_job_{TIMESTAMP}"
BASELINE_DATASET = ml_monitoring.spec.MonitoringInput(
  gcs_uri=FEATURE_ATTRIBUTION_BASELINE_DATASET, data_format="jsonl"
)
TARGET_DATASET = ml_monitoring.spec.MonitoringInput(
  batch_prediction_job=batch_prediction_job_2.resource_name
)
model_monitoring_job_3 = my_model_monitor.run(
  display_name=JOB_DISPLAY_NAME,
```

```
  baseline_dataset=BASELINE_DATASET,
  target_dataset=TARGET_DATASET,
  tabular_objective_spec=ml_monitoring.spec.TabularObjective(
    # Feature attribution spec.
    feature_attribution_spec=FEATURE_ATTRIBUTION_SPEC
  ),
  # You must have a Explanation spec for feature attribution monitoring.
  # You can specify the explanation spec in the Model, Model monitor, or the Model
monitoring job.
  explanation_spec=EXPLANATION_SPEC,
  notification_spec=NOTIFICATION_SPEC,
  output_spec=OUTPUT_SPEC,
)
```

This code configures and runs a **Feature Attribution Monitoring** job. This process requires Vertex AI to "understand" the structure of your model (via ExplanationMetadata) so it can calculate *which features* contributed most to the model's predictions and monitor if those contributions change over time.

1. Defining the Explanation Specification

This block acts as the "map" for the AI, telling it how to interpret the model's architecture.

- **from ... import (ExplanationMetadata, ...)**: Imports the classes needed to map your model's inputs to the Explainable AI (XAI) service.
- **EXPLANATION_SPEC = ExplanationSpec(...)**: Defines the global configuration for generating explanations.
- **parameters=ExplanationParameters({"sampled_shapley_attribution": {"path_count": 2}})**: Sets the algorithm. **Sampled Shapley** is a method used to approximate how much each feature contributed to a prediction by calculating the average marginal contribution of a feature across different combinations. The path_count controls the approximation complexity (higher values are more accurate but slower).
- **metadata=ExplanationMetadata(...)**: This massive block is the **Schema Mapping**.
  - **inputs={...}**: This maps every single input feature your model uses (e.g., cnt_ad_reward, country). For each feature, it defines:
    - input_tensor_name: The exact name the model expects.
    - encoding: IDENTITY means use the raw value.
    - modality: Defines if it is numeric (math) or categorical (labels).
  - **outputs={...}**: Identifies which output node (e.g., churned_probs) the monitor should calculate attributions for.

---

2. Preparing the Baseline Data

To monitor attribution drift, you need a "reference" or "baseline" dataset to compare against production data.

- **FEATURE_ATTRIBUTION_BASELINE_DATASET = ...**: Defines the path for your reference file.
- **! gsutil cp ...**: Downloads a sample dataset (specifically one without ground truth) from a Google-hosted bucket to your local project bucket, to use as the standard for what "normal" attributions look like.

---

3. Executing the Monitoring Job

This code links the configuration to the actual execution.

- **import pandas as pd...**: Generates a current timestamp to ensure the job name is unique.
- **BASELINE_DATASET = ...**: Sets the reference data (the file we just downloaded) as the baseline_dataset. The monitor will calculate the "normal" feature attributions using this file.
- **TARGET_DATASET = ...**: Points to batch_prediction_job_2. This is the production data we want to "interrogate" to see if the model's reasoning has changed.
- **model_monitoring_job_3 = my_model_monitor.run(...)**: Starts the job.
    - **tabular_objective_spec**: Activates the feature_attribution_spec (which we defined in a previous code block) to perform the math.
    - **explanation_spec=EXPLANATION_SPEC**: **This is the critical link.** It injects the metadata mapping we defined in Step 1. Without this, the monitor wouldn't know how to "read" the model's tensors.
    - **notification_spec & output_spec**: Standard settings to ensure you get alerted and the logs are saved to your GCS bucket.

Summary of what just happened:

You have provided Vertex AI with a **schema** (EXPLANATION_SPEC) that tells it how to disassemble your model's decisions. You then told it to:

1. Calculate how the model "thinks" using the BASELINE_DATASET.
2. Calculate how the model "thinks" using the TARGET_DATASET (real-world data).
3. Compare those two "thoughts" and alert you if the model's reliance on specific features (like cnt_ad_reward) has shifted significantly.

Check monitoring metrics: Cloud Storage bucket

Run the following to view Model Monitoring metrics stored in the Cloud Storage bucket.

```
try:
  my_model_monitor.show_feature_drift_stats(model_monitoring_job_1.name)
except Exception as e:
  print(e)
try:
  my_model_monitor.show_feature_drift_stats(model_monitoring_job_2.name)
except Exception as e:
  print(e)
try:
  my_model_monitor.show_output_drift_stats(model_monitoring_job_2.name)
except Exception as e:
  print(e)
```

This code is used to retrieve and display the statistical results generated by your Model Monitoring jobs. Since these jobs run as asynchronous processes in the cloud, this code is wrapped in try/except blocks to handle instances where a job might still be running or might have failed, preventing your entire notebook from crashing.

Here is the line-by-line explanation:

- **try:**: Initiates a block of code to attempt an action. If the action succeeds, it continues normally; if an error occurs, the code jumps to the except block.
- **my_model_monitor.show_feature_drift_stats(model_monitoring_job_1.name)**: This command fetches the calculated **feature drift** statistics from the first monitoring job (job_1). It essentially goes to your Cloud Storage bucket, parses the JSON/log files, and renders a visual report (usually a table or chart) directly in your Jupyter notebook.
- **except Exception as e:**: This catches any errors that occurred during the previous line (e.g., the job is still "running," the file hasn't been written yet, or the job failed).
- **print(e)**: If an error occurred, this prints the error message so you can understand *why* the data wasn't retrieved, rather than just seeing the notebook stop silently.
- **try: ... my_model_monitor.show_feature_drift_stats(model_monitoring_job_2.name)**: The process is repeated for job_2. This allows you to compare the feature drift stats between your two different batch prediction runs.
- **except ... print(e)**: Again, handles any potential errors for the second retrieval attempt.

- **try: ... my_model_monitor.show_output_drift_stats(model_monitoring_job_2.name)**: This is a distinct command that focuses on **prediction output drift** (how the model's actual decisions changed) rather than input feature drift.
- **except ... print(e)**: Finally, handles errors for this specific retrieval.

Why this is helpful:

Instead of manually navigating through your Google Cloud Storage bucket and trying to parse complex JSON log files yourself, these helper methods extract that data and format it into a readable report for you to inspect in your notebook environment.